



Jose Parreño Garcia <joparga3@gmail.com>

[TEST d6a - Paid] DS-STAR: How Google built a Data Science agent that actually works

1 mensaje

Jose Parreño Garcia from Senior Data Science Lead <joseparreogarcia@substack.com>
Para: joparga3@gmail.com

7 de abril de 2026 a las 16:03

[View in browser](#)

The Senior Data Science Lead newsletter

Hi! It's Jose from Senior Data Science Lead newsletter, where I write about senior data science leadership, sharing hard-earned lessons from building teams, systems, and real-world machine learning.

Because you are a paid subscriber, you are directly supporting this work — and you get full access to the practical tools built alongside the writing:

- *The full Data Storytelling course, focused on turning analysis into insights that land with executives. Because today, anyone can build a chart with AI, but very few people can build the right chart.*



Master Data Storytelling

JOSE PARREÑO GARCIA · NOVEMBER 27, 2025

[Read full story](#)

- *50+ Notion templates for hiring, onboarding, mentoring, and planning built for Data and Tech Leads. As a paid subscriber, you get **50% off** with code **PAIDSUBS-03PL**.*



50+ Data Science Templates

JOSE PARREÑO GARCIA · APRIL 8, 2025

[Read full story](#)

And as always, thank you for supporting the newsletter — your support is what keeps it independent, practical, and focused on real-world impact.

DS-STAR: How Google built a Data Science agent that actually works

How Google's seven-module pipeline beats raw Gemini by 32 percentage points — and what that means for anyone building data science agents.

JOSE PARREÑO GARCIA
APR 7 · DRAFT





READ IN APP ↗

I have been using agentic coding tools for about two years (Claude Code for the past year, consistently and seriously). It is genuinely impressive at general-purpose engineering tasks. But I kept running into a friction point whenever I pushed it toward real data science. Claude's analysis would mostly stay at a few averages, which were actually useful, but missing the depth I would expect from a Data Scientist (distribution comparisons, p-values, confidence intervals, data quality, etc).

It felt like asking a generalist to do specialist work, useful, but not top level. So, I went looking. And this week I found the paper I had been waiting for.

[DS-STAR is a Data Science Agent for Solving Diverse Tasks by Google](#). The central argument of the paper is that the power comes from the framework and system design, not by the power of new frontier models. As a teaser:

- Gemini 2.5 Pro alone scores 12.70% on hard-level DABStep tasks.
- DS-STAR with Gemini 2.5 Pro scores 45.24%.
- That is a 32-percentage-point jump.
- The model did not change. **The system around it did.**

That framing resonates with me. Claude Code outperforms other AI coding tools not because it always has the best frontier model, but because of how the system is wired. DS-STAR applies the same logic to data science. And for that, I wanted to really deep dive into the system. I break it down here for you.

 Subscribed

Where can you find the paper?

[DS-STAR: Data Science Agent for Solving Diverse Tasks across Heterogeneous Formats and Open-Ended Queries](#)

This paper was submitted to arXiv in September 2025, revised February 2026 (v4). Written by Jaehyun Nam (KAIST), Jinsung Yoon, Jiefeng Chen, Raj Sinha, and Tomas Pfister (Google Cloud). If you are interested in a condensed version, [Google also released a webpage](#).

The paper presents DS-STAR, a Data Science “*agent designed to (1) seamlessly process and integrate data across diverse, heterogeneous formats, and (2) move beyond simple QA to generate comprehensive research reports for open-ended queries.*”

What will this blog post cover?

- **Goal of the paper** — what DS-STAR is trying to solve and why existing single-pass approaches fail on real data science tasks
- **How DS-STAR is structured** — the 2-component architecture: DS-STAR for well-defined questions and DS-STAR+ for open-ended research

- **DS-STAR deep dive** — the 7 modules, their formulas, and the full iterative algorithm
- **DS-STAR+ deep dive** — how the decomposition, writer, and refinement loop extend the inner engine to produce structured reports
- **The prompts behind the agent** — what Google actually published in Appendix L and what the design choices reveal
- **Ablation tests** — which component contributes most to the 32-point performance gain
- **More rounds for harder problems** — how the iteration count scales with task complexity
- **Google's example report** — what DS-STAR+ actually produces on a real payments dataset
- **Limitations** — no official release, open integration questions, and others.

Goal of the paper

The stated goal is ambitious: **build a system that can answer *any* data science question across heterogeneous data formats and open-ended queries.**

What makes that hard in practice is the data. Real data science work rarely involves a single clean CSV and a well-specified question. It involves **multiple structured datasets requiring multi-step reasoning**, domain-specific knowledge baked into the rules, and questions that cannot be answered with

a single code generation pass. The DABStep benchmark from Adyen and Hugging Face captures this well: 450 tasks, 72 easy (single dataset, minimal contextual knowledge), and 378 hard (multiple datasets, domain knowledge required, no single-shot solution). The [KramaBench benchmark](#) extends this further: 104 challenges spanning 1,700 files and 24 data sources across 6 domains, where agents must autonomously discover which files are even relevant.

Both benchmarks are designed to break the approach of “write one code block and submit the answer.” That is exactly the approach DS-STAR replaces.

Even if your actual use case is narrower — a single internal database, a well-defined question — the architecture choices here still hold. A system that handles heterogeneous chaos will handle clean data just fine. The reverse is not true.

How the DS-STAR data science agent is structured

DS-STAR is actually two things, built one on top of the other.

Created with Datawrapper

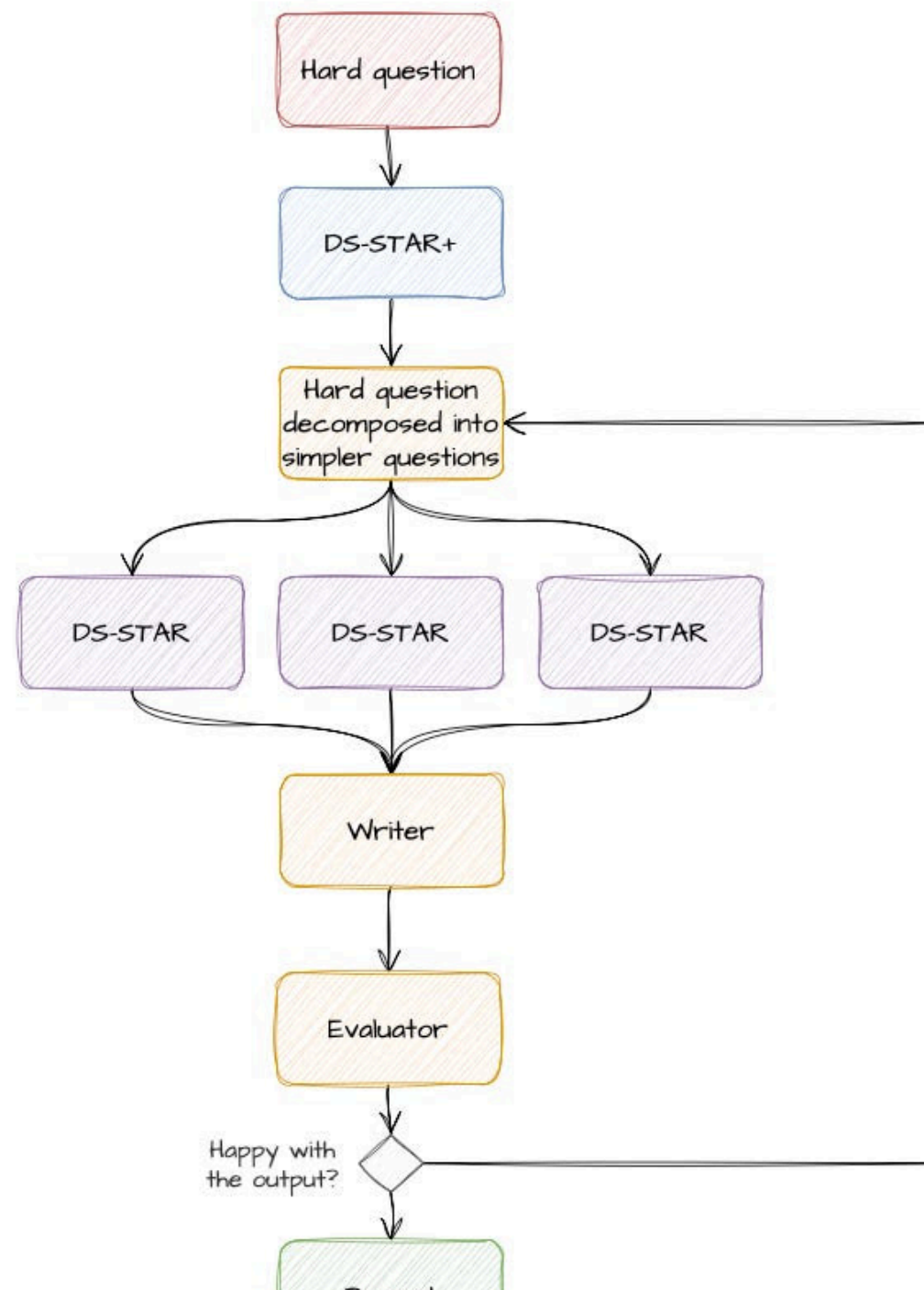
Component	Input	Output	Evaluation benchmark
DS-STAR	Well-defined question + data files	Single answer	DABStep, KramaBench, DA-Code
DS-STAR+	Open-ended research query + data files	Multi-section data science report	DABStep-Research

Created with Datawrapper

Architecturally, DS-STAR is the inner engine of DS-STAR+.

- DS-STAR+ decomposes the open-ended query into a set of focused sub-questions, hands each to DS-STAR.
- Then, uses a writer agent to synthesise the answers into a report.
- An evaluator also exists to generate more rounds of analysis.

Below you can find an overly-simplified of how DS-STAR+ and DS-STAR interact.

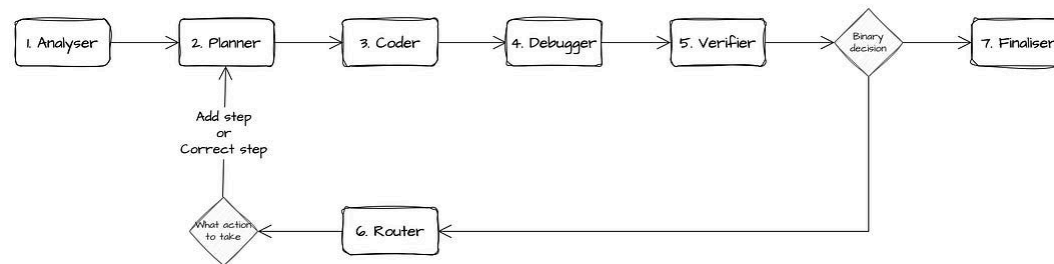




DS-STAR deep dive

Whilst the diagram in the paper is pretty good, I feel it's missing a couple of components to follow the sequence of the algorithm better. Below is my attempt at improving it.

DS-STAR is made up of 7 modules (and a self-refinement loop)



3.1. The seven modules.

DS-STAR comprises seven named agents. Walking them in pipeline order makes the system legible.

Module 1. The ANALYSER.

The first thing DS-STAR does before any planning is **profile every data file it receives**. It generates a Python script for each file, executes it, and stores

the printed output as a structured description. Think of this like an indexer or a small RAG.

I also found pretty interesting that the description is generated deterministically via Python, instead of letting an LLM decide how to analyse the file. Below you can find 2 screenshots: (1) Sample python code to analyse a json file (2) The output of this script on a sample file.

I. Qualitative results on the data file analyzer agent

I.1. Code generated by the analyzer agent

```
import json

def describe_json_data(file_path):
    """
    Loads a JSON file, prints its full content, and then describes its structure
    and essential information, including column names (keys) and example data.
    """
    # Load the JSON data from the file
    # Using utf-8 encoding is a good practice for JSON.
    with open(file_path, 'r', encoding='utf-8') as f:
        data = json.load(f)

    print(f"--- Describing content of: {file_path} ---")
    print("\n")

    # 1. Print the full loaded content as per requirement
    # "The Python code should print out the content of data/merchant_data.json."
    print("--- Full Loaded Content ---")
    # Using json.dumps for a pretty-printed representation of the loaded Python object.
    # ensure_ascii=False allows for proper display of unicode characters.
    print(json.dumps(data, indent=4, ensure_ascii=False))
    print("-" * 50)
    print("\n")

    # 2. Print Essential Information
    print("--- Essential Information ---")
    data_type = type(data)
    print(f"Overall Data Type: {data_type.__name__}")

    if isinstance(data, dict):
        print("The data is a single JSON object (dictionary).")
```

Page 50 of the paper. Sample python script to analyse a json file.

I.2. Examples of generated data descriptions

```
f'''
File: data/merchant_data.json (6857 bytes)
Top-level type: array
root: array with 30 items
Element types: object=30
Columns (5): ['account_type', 'acquirer', 'capture_delay', 'merchant',
→ 'merchant_category_code']
Sample rows (first 5):
[
  {
    "merchant": "Crossfit_Hanna",
    "capture_delay": "manual",
    "acquirer": [
      "gringotts",
      "the_savings_and_loan_bank",
      "bank_of_springfield",
      "dagoberts_vault"
    ],
    "merchant_category_code": 7997,
    "account_type": "F"
  }
]
```

Page 51 of the paper. Results coming out of the sample python script to analyse a json file.

Google also kindly shares their prompts for each module, here is the prompt for the analyser module.

You are an expert data analyst.

Generate a Python code that loads and describes the content of {filename}.

Requirement

- The file can both unstructured or structured data.
- If there are too many structured data, print out just few examples.
- Print out essential informations. For example, print out all the column names.
- The Python code should print out the content of {filename}.

- The code should be a single-file Python program that is self-contained and can be executed as-is.
- Your response should only contain a single code block.
- Important: You should not include dummy contents since we will debug if error occurs.
- Do not use try: and except: to prevent error. I will debug it later.

Module 2. The PLANNER.

Given the data descriptions and the user's question, the Planner generates the first step of an execution plan. **Not the full plan at once, *just one step*.**

The Planner has two modes: `planner_init` (first step, no prior plan context) and `planner_next` (subsequent steps, given current plan and prior execution results).

Below is the prompt for initialising a plan:

You are an expert data analyst.
In order to answer factoid questions based on the given data, you have to first plan effectively.

Question

{question}

Given data: {filenames}

```
{filenames #1}  
{summaries #1}  
...  
{filenames #N}  
{summaries #N}
```

Your task

- Suggest your very first step to answer the question above.
- Your first step does not need to be sufficient to answer the question.
- Just propose a very simple initial step, which can act as a good starting point to answer the question.
- Your response should only contain an initial step.

When the system is beyond the initial plan, and planning a next step, here is the adjusted prompt. Here, it is very interesting to see how the prompt includes previous step plans and the result from the previous plan. This is way to give the system memory of what was tried before for in-depth context.

You are an expert data analyst.

In order to answer factoid questions based on the given data, you have to first plan effectively.

Your task is to suggest next plan to do to answer the question.

Question

{question}

Given data: {filenames}

{filenames #1}

{summaries #1}

...

{filenames #N}

{summaries #N}

Current plans

1. {Step 1}

...

k. {Step k}

Obtained results from the current plans:

{result}

Your task

- Suggest your next step to answer the question above.
- Your next step does not need to be sufficient to answer the question, but if it requires only final simple last step you may suggest it.
- Just propose a very simple next step, which can act as a good intermediate point to answer the question.
- Of course your response can be a plan which could directly answer the question.
- Your response should only contain an next step without any explanation.

Module 3. The CODER.

Takes the current plan step and generates executable Python code to implement it. Like the Planner, it has two modes: `coder_init` (first code block) and `coder_next` (subsequent blocks, with awareness of prior results). The code is generated as a single self-contained script.

The prompt for the initial coder is the following:

```
# Given data:
{filenames}
{filenames #1}
{summaries #1}
...
{filenames #N}
{summaries #N}

# Plan
{plan}

# Your task
- Implement the plan with the given data.
- Your response should be a single markdown Python code (wrapped
in ```).
- There should be no additional headings or text in your
response.
```

The prompt for coding the next action is the following.

You are an expert data analyst.

Your task is to implement the next plan with the given data.

Given data:

```
{filenames}  
{filenames #1}  
{summaries #1}  
...  
{filenames #N}  
{summaries #N}
```

Base code

```
```python  
{base_code}
```
```

Previous plans

```
1. {Step 1}  
...  
k. {Step k}
```

Current plan to implement

```
{Step k+1}
```

Your task

- Implement the current plan with the given data.
- The implementation should be done based on the base code.
- The base code is an implementation of the previous plans.
- Your response should be a single markdown Python code (wrapped


```
in ```).
```

— There should be no additional headings or text in your response.

Module 4. The DEBUGGER.

The Coder's output runs against the actual data files.

If execution fails (a syntax error, a missing column, a type mismatch) the Debugger fires. It takes the failed code, the error message, and generates a corrected script.

The Debugger is not shown prominently in Figure 1 of the paper, but it is in the algorithm: it sits between the Coder and the Verifier, handling execution failures before they even reach plan evaluation. This is why I purposely added it in my own diagram.

The Debugger has 2 prompts:

1. A prompt to summarise the error (because we know how long traceback errors can be).
2. A prompt to actually fix the bug.

Below you can see both.

```
# --> PROMPT TO SUMMARISE THE ERROR
```

```
# Error report
```

```
{bug}
```

Your task

- Remove all unnecessary parts of the above error report.
- We are now running {filename}.py. Do not remove where the error occurred.

--> PROMPT TO FIX THE ERROR**# Code with an error:**

```
```python  
{code}
```
```

Error:

```
{bug}
```

Your task

- Please revise the code to fix the error.
- Provide the improved, self-contained Python script again.
- There should be no additional headings or text in your response.
- Do not include dummy contents since we will debug if error occurs.
- All files/documents are in ``data/`` directory.

Module 5. The VERIFIER.

After the code runs successfully, the Verifier assesses whether the current plan is sufficient to answer the original question.

The verdict v is binary: `sufficient` or `insufficient`.

The evaluation is based on the cumulative plan, the user's query, the solution code, which is an implementation of the cumulative plan, and its execution result, using the prompt below.

You are an expert data analyst.

Your task is to check whether the current plan and its code implementation is enough to answer the question.

Plan

1. {Step 1}

...

k. {Step k}

Code

```
```python
{code}
```
```

Execution result of code

{result}

Question

{question}

Your task

– Verify whether the current plan and its code implementation is

enough to answer the question.

- Your response should be one of 'Yes' or 'No'.
- If it is enough to answer the question, please answer 'Yes'.
- Otherwise, please answer 'No'.

Module 6. The ROUTER.

Triggered only when the Verifier returns *insufficient*.

The Router looks at the current plan, the question, the execution results, and makes a binary decision:

- Option 1. Does the plan need a new step added at the end ("Add Step")?
- Option 2. Or, does a specific existing step need to be corrected ("Replace Step K", where K is the step index)?

Option 1 is logical in the sense that maybe the question is incomplete. But importantly, because we are going 1 step at a time, option 2 can help the system easily correct 1 single step, not having to re-write the *whole* plan again.

You are an expert data analyst.

Since current plan is insufficient to answer the question, your task is to decide how to refine the plan to answer the question.

Question

{question}

Given data:

{filenames}

{filenames #1}

{summaries #1}

...

{filenames #N}

{summaries #N}

Current plans

1. {Step 1}

...

k. {Step k}

Obtained results from the current plans:

{result}

Your task

- If you think one of the steps of current plans is wrong, answer among the following options: Step 1, Step 2, ..., Step K.
- If you think we should perform new NEXT step, answer as 'Add Step'.
- Your response should only be Step 1 – Step K or Add Step.

Module 7. The FINALISER.

When the Verifier returns `sufficient`, the Finaliser synthesises all execution outputs into the final answer. It extracts the relevant result from the accumulated execution history and formats it as the response to the original question.

You are an expert data analyst.

You will answer factoid question by loading and referencing the files/documents listed below. You also have a reference code.

Your task is to make solution code to print out the answer of the question following the given guideline.

Given data: {filenames}

{filenames #1}

{summaries #1}

...

{filenames #N}

{summaries #N}

Reference code

```
```python{code}
```

```
```
```

Execution result of reference code

{result}

Question

{question}

Guidelines

{guidelines}

Your task

- Modify the solution code to print out answer to follow the give guidelines.
- If the answer can be obtained from the execution result of the reference code, just generate a Python code that prints out the desired answer.
- The code should be a single-file Python program that is self-contained and can be executed as-is.
- Your response should only contain a single code block.
- Do not include dummy contents since we will debug if error occurs.
- Do not use try: and except: to prevent error. I will debug it later.
- All files/documents are in ``data/`` directory.

Important highlight

The pipeline is sequential, not parallel.

Each agent's output is the next agent's input. Changing any one of them changes what the next one sees.

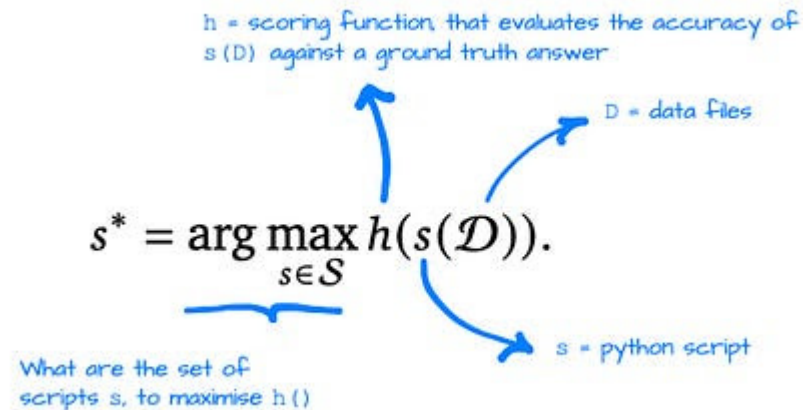
3.2. The formulas.

One really interesting thing from the paper is how Google parametrises the algorithm into formulas. This isn't a, *"here is how all the prompts work together because we are smart humans and we figured this out"*. It's more a, *"hey, this system has logical steps, and we can describe it clearly with formulas"*.

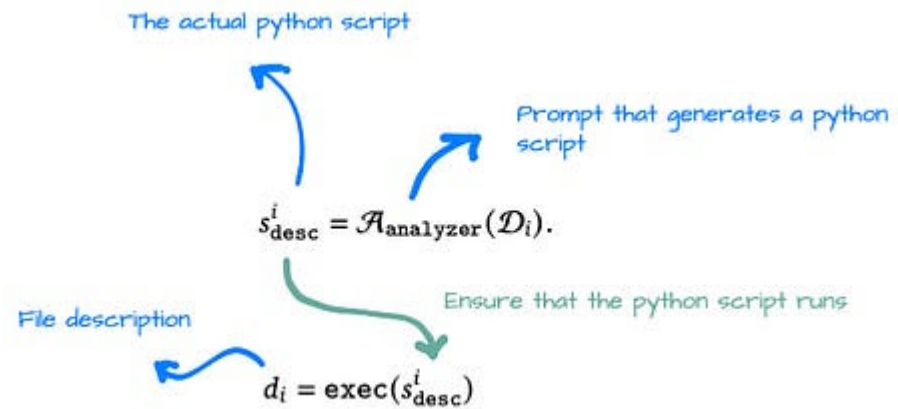
These formulas can help the Google folks adapt each prompt, each python template, each script to fine-tune performance. They don't mention this clearly, but I assume this is how the formulas were used when coming up with the final DS-STAR system design.

Let's go through them first and then map them to the flow diagram too.

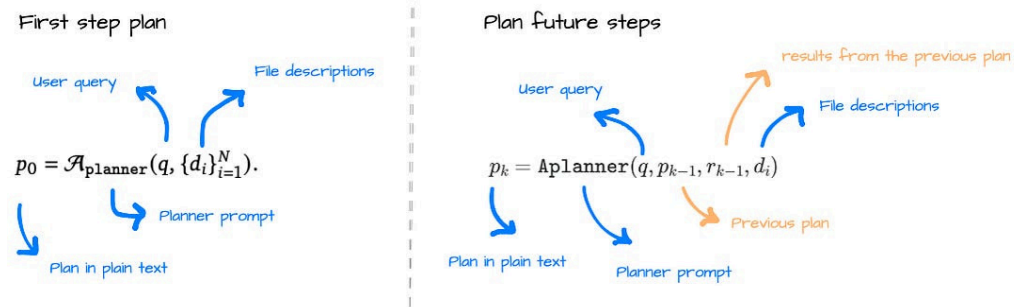
0. Problem setup.



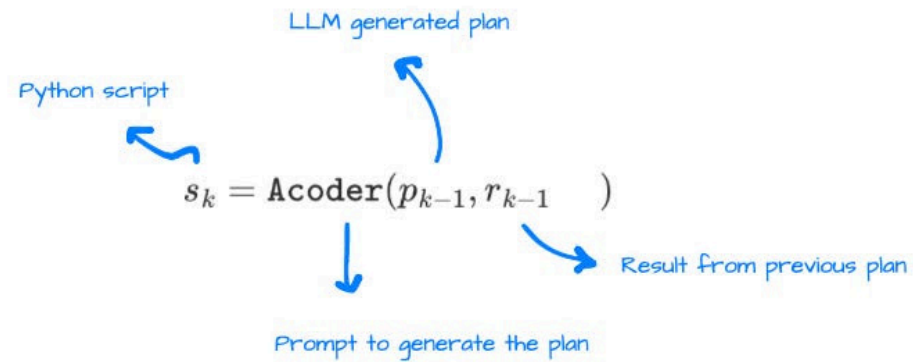
1. The analyser.



2. The planner.

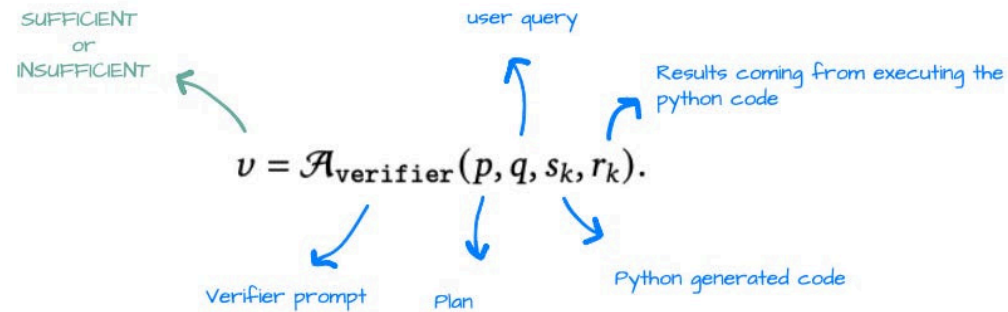


3. The coder.

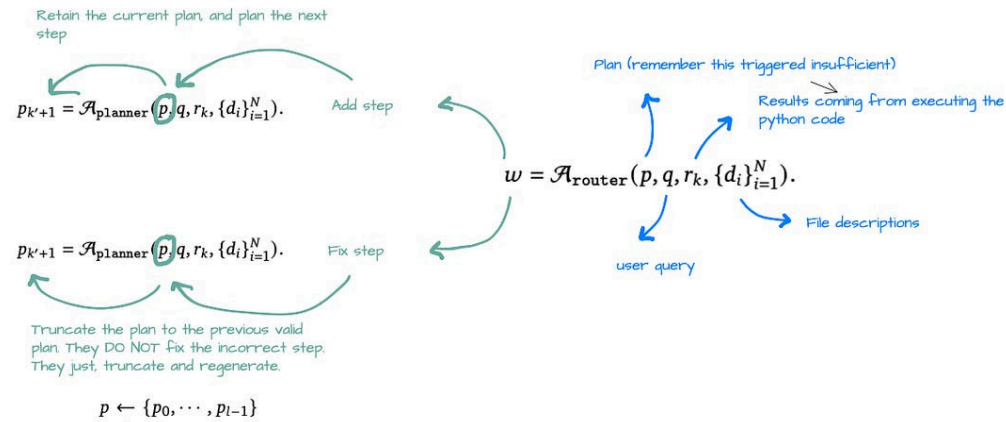


4. **The debugger.** No formulas, as this is a simple steps of capturing any debugging issues and fixing them. It doesn't change the fact that the output is a python code that works.

5. **The verifier.**



6. **The router.**



7. **The Finaliser.** Again, no formula really to show. This step is a prompt to write the final answer for a detailed question (say a table, or a number, or sometimes a summarised report).

3.3. The algorithm.

Finally, I'd like to cover the actual pseudo-code for the whole DS-STAR flow. If you have read the previous sections, with modules and formulas in place, Algorithm 1 is a straightforward read.

C.1. Algorithm of DS-STAR

Algorithm 1 DS-STAR

```

1: Input: query  $q$ , data files  $\mathcal{D}$ , maximum number of refinement round  $M$ 
2: # Analyzing data files
3: for  $i = 1$  to  $N$  do
4:    $s_{desc}^i = \mathcal{A}_{analyzer}(\mathcal{D}_i)$ 
5:    $d_i = \text{exec}(s_{desc}^i)$ 
6: end for
7: # Iterative plan generation and verification
8:  $p_0 = \mathcal{A}_{planner}(q, \{d_i\}_{i=1}^N)$ 
9:  $s_0 = \mathcal{A}_{coder}(p_0, \{d_i\}_{i=1}^N)$ 
10:  $r_0 = \text{exec}(s_0)$ 
11:  $p = \{p_0\}$ 
12: for  $k = 0$  to  $M - 1$  do
13:    $v = \mathcal{A}_{verifier}(p, q, s_k, r_k)$ 
14:   if  $v = \text{sufficient}$  then
15:     break
16:   else if  $v = \text{insufficient}$  then
17:      $w = \mathcal{A}_{router}(p, q, r_k, \{d_i\}_{i=1}^N)$ 
18:     if  $w \in \{0, \dots, \text{len}(p) - 1\}$  then
19:        $l = w - 1$ 
20:     else if  $w = \text{Add Step}$  then
21:        $l = k$ 
22:     end if
23:      $p \leftarrow \{p_0, \dots, p_l\}$ 
24:      $p_{l+1} = \mathcal{A}_{planner}(p, q, r_k, \{d_i\}_{i=1}^N)$ 
25:      $p \leftarrow \{p_0, \dots, p_l, p_{l+1}\}$ 
26:      $s_{k+1} = \mathcal{A}_{coder}(p, q, s_k, \{d_i\}_{i=1}^N)$ 
27:      $r_{k+1} = \text{exec}(s_{k+1})$ 
28:   end if
29: end for
30: Output: Final solution  $s$ 

```

For every file, run the analyser and store the description d_i
 Generate the first plan
 Generate the python code based on p_0
 Execute the python code s_0 and extract the result (reminder, the Debugger fires here in case of an error)
 The verifier evaluates the cumulative plan k
 If insufficient, the router decides whether to add a step or replace step K
 Replace step
 Keep step
 Re-run the planning, coding and execution step with new plan based on the router step
 The finaliser extracts the final answer from accumulated results.

In simplified words, it would be something like.

Phase 1 — Data profiling. For every file in $\mathcal{D}$, run the Analyzer and store the description d_i . This is the foundation everything else builds on.

Phase 2 — Iterative plan generation.

1. The Planner generates the first plan step from the question and the data descriptions.
2. The Coder generates code for that step.

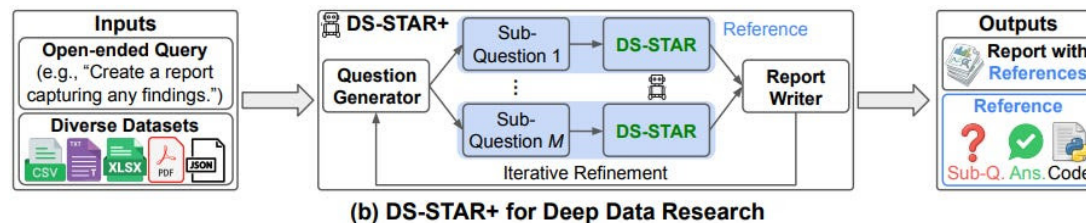
3. The Executor runs it. If it fails, the Debugger corrects the code and re-runs.
4. The Verifier evaluates the cumulative plan against the execution output. If sufficient, jump to step 6.
5. If insufficient, the Router decides whether to add a new step or replace step K. Return to step 1 (Planner) with updated context. Repeat up to the maximum iteration limit.
6. The Finalyzer extracts the final answer from accumulated results.

The algorithm reads like a feedback control loop (I come from robotics engineering, so I always think about PID controllers).

- The Verifier is the sensor.
- The Router is the controller.
- The Planner and Coder are the actuators.
- The loop terminates when the sensor signals “done.”

DS-STAR+ deep dive

As we mentioned earlier, DS-STAR+ takes an open-ended query to build a structured report from it. The diagram below shows how the enhanced system operates. In this case, I did think the official diagram was detailed enough to share (without having to build one myself).



Lets break the diagram down:

1. **The first problem is decomposition.** A vague query cannot be handed to DS-STAR directly. DS-STAR+ adds a GENERATOR agent that reads the query and the data descriptions and produces a set of focused sub-questions $\{f_i\}$. Each sub-question is answerable by DS-STAR: it has a defined question and a reference to the relevant data. DS-STAR runs on each one independently and returns answers $\{a_i\}$.
2. **The WRITER agent then synthesises those sub-question/answer pairs into a structured report R.** The prompt instructs the Writer to cite each claim back to the sub-question that grounded it (attempting to prevent hallucination by making every statement traceable to an executed data query).
3. **The refinement loop is similar to DS-STAR.** The Generator re-examines the current draft report alongside the data descriptions and generates a new set of sub-questions targeting what is missing or shallow.

4.2. The algorithm

The DS-STAR+ notation extends the DS-STAR chain with a decomposition layer. I will not cover the exact formulas in the paper, but I will jump to the

algorithm to see how the loop operates and what inputs flow through.

C.2. Algorithm of DS-STAR+

Algorithm 2 DS-STAR+

```

1: Input: query  $q$ , data files  $\mathcal{D}$ , maximum number of refinement round  $K$ 
2: # Analyzing data files
3: for  $i = 1$  to  $N$  do
4:    $s_{\text{desc}}^i = \mathcal{A}_{\text{analyzer}}(\mathcal{D}_i)$ 
5:    $d_i = \text{exec}(s_{\text{desc}}^i)$ 
6: end for
7: # Initial sub-question generation
8:  $\{f_i^0\}_{i=1}^{M_0} = \mathcal{A}_{\text{generator}}(q, \{d_i\}_{i=1}^N)$ 
9:  $\{a_i^0\}_{i=1}^{M_0} = \text{DS-STAR}(\{f_i^0\}, \mathcal{D})$ 
10: # Initial report writing
11:  $R = \mathcal{A}_{\text{writer}}(q, \{f_i^0, a_i^0\}_{i=1}^{M_0})$ 
12: # Iterative refinement
13: for  $k = 1$  to  $K$  do
14:    $\{f_i^k\}_{i=1}^{M_k} = \mathcal{A}_{\text{generator}}(q, \{d_i\}_{i=1}^N, R)$ 
15:    $\{a_i^k\}_{i=1}^{M_k} = \text{DS-STAR}(\{f_i^k\}, \mathcal{D})$ 
16:    $R \leftarrow \mathcal{A}_{\text{writer}}(q, R, \{f_i^k, a_i^k\}_{i=1}^{M_k})$ 
17: end for
18: Output: Final data science report  $R$ 

```

Diagram annotations:

- Lines 4-5: For every file, run the analyser and store the description d_i (same as DS-STAR)
- Lines 8-9: Decompose the query into sub-questions; DS-STAR answers each sub-question
- Line 11: Write compiles an initial report
- Lines 14-16: Refinement loops, where the generator receives not only the documents and query (same as line 8), but also the report R from line 11.

4.3. The prompts behind DS-STAR+

We covered the relevant DS-STAR prompts in the previous section together with the formula decomposition. We didn't cover the formulas for DS-STAR+, but I still want to share the prompts used for this loop.

Sub-question generator agent (first pass)

You are an expert data analyst.

Your task is to write a comprehensive data science report to the given question by using the files/documents listed below.

In order to do this, you have to first suggest multiple data analysis questions that should be answered to write the report.

Given data: {filenames}

{filenames #1}

{summaries #1}

...

{filenames #N}

{summaries #N}

Question

{question}

Your task

- Suggest multiple factoid data analysis questions that are required to write the report really well.
- All the questions should be well-answered using the given data.
- All questions should be answered independently.
- Generate as much as you can.
- Return in valid JSON format:
Questions = {'question': str}
Return: list[[Questions](#)]

Sub-question generator agent (refinement pass)

You are an expert data analyst.

Your task is to complement the given data science report of the given question.

In order to do this, you have to suggest supplementary multiple data analysis questions that can strengthen to the report.

Given data: {filenames}

{filenames #1}

{summaries #1}

...

{filenames #N}

{summaries #N}

Given data science report:

{report}

Question

{question}

Your task

- Suggest multiple factoid data analysis questions that are required to complement the report.
- All questions should contain new information that is not included in the report.
- All the questions should be well-answered using the given data.
- All questions should be answered independently.
- Return in valid JSON format:

```
Questions = {'question': str}
Return: list[Questions]
```

Writer agent (first generated report)

You are an expert data analyst.

Your task is to write a ****comprehensive data science report**** to the given question by using the data and some relevant informations listed below.

Relevant informations:

{Sub-Question #1}

{Answer #1}

...

{Sub-Question #M_0}

{Answer #M_0}

Question that you have to write a comprehensive data science report:

{question}

Your task:

- The report should be grounded to the given relevant informations.
- For the citation, use the Sub-Question number as a citation number which is in $1 - \{\text{len}(\text{subquestions})\}$.
- The data science report should be relevant to given question, should be comprehensive,

and should be insightful.

- The data science report should have nice structure, good readability, and should be professional.
- Write a very comprehensive data science report to the given above question.

Writer agent (refined report)

You are an expert data analyst.

Your task is to complement the given data science report of the given question by using the some relevant informations listed below.

Relevant informations:

{Sub-Question #1}

{Answer #1}

...

{Sub-Question #M_k}

{Answer #M_k}

Given data science report:

{report}

Question that you have to write a comprehensive data science report:

{question}

Your task:

- Do not modify the given report a lot. Just try to add new information.
- The report should be grounded to the given relevant informations.
- Cite with alphabet. For the citation, use the Sub-Question number as a citation alphabet (e.g., cite with [a] for the Sub-Question 1).
- The data science report should be relevant to given question, should be comprehensive, and should be insightful.
- The data science report should have nice structure, good readability, and should be professional.
- Complement the give data science report to the given above question.

For readers who want to run the system: [JulesLscx/DS-Star](#) is a faithful community re-implementation (145 stars, 37 forks as of the research date). It contains `dsstar.py` (the full `DS_STAR_Agent` class), `prompt.yaml` (all prompts in text form), and `provider.py` (Gemini, OpenAI, Ollama). This is not the official Google release — there is none — but it is an accurate implementation of the paper.

Now that we covered how the system works, I'd like to highlight some interesting scientific approaches on evaluating how good the system is.

Ablation tests

Table 4 in the paper answers the question: which component contributes most to DS-STAR’s performance? The setup is ablation by removal: strip out one component, measure the drop.

Table 4 | **Ablation study on each component.** We report accuracy (%) on easy and hard-level tasks in DABStep benchmark.

| Analyzer
$\mathcal{A}_{\text{analyzer}}$ | Verifier
$\mathcal{A}_{\text{verifier}}$ | Router
$\mathcal{A}_{\text{router}}$ | Easy | Hard |
|---|---|---|-------|-------|
| ✗ | ✓ | ✓ | 75.00 | 26.98 |
| ✓ | ✗ | ✓ | 83.33 | 34.66 |
| ✓ | ✓ | ✗ | 79.17 | 39.95 |
| ✓ | ✓ | ✓ | 87.50 | 45.24 |

Remove the Analyzer. Hard-level DABStep accuracy drops from 45.24% to 26.98%. *The single largest drop of any component removal. Without the data descriptions, the Planner is generating steps with no grounding in what the data actually contains.* The performance is still better than the 12.70% baseline (a non-agentic framework), which shows the other components still contribute, but the Analyzer is by far the most load-bearing module.

Remove the Router (force “Add Step” only). Performance drops on both easy and hard tasks. The Router’s ability to replace a flawed step — rather than just appending new steps to a broken plan — is what prevents the system from compounding errors. Without it, the plan (and errors it has) accumulates.

Replace step-by-step VERIFIER with full-plan-then-execute. The baseline here is: generate the entire plan at once, run all the code, use code execution success as the only verification signal. This performs worse than DS-STAR's step-by-step approach. The step-by-step Verifier catches plan problems at the level of individual steps, before they propagate to the next one.

Data scientists will recognise this framing immediately: this is feature importance analysis for an agent system. Strip out each component and measure the degradation.

1. The Analyzer is the highest-importance feature.
2. The Router is second.
3. The step-by-step Verifier outperforms the full-plan baseline.

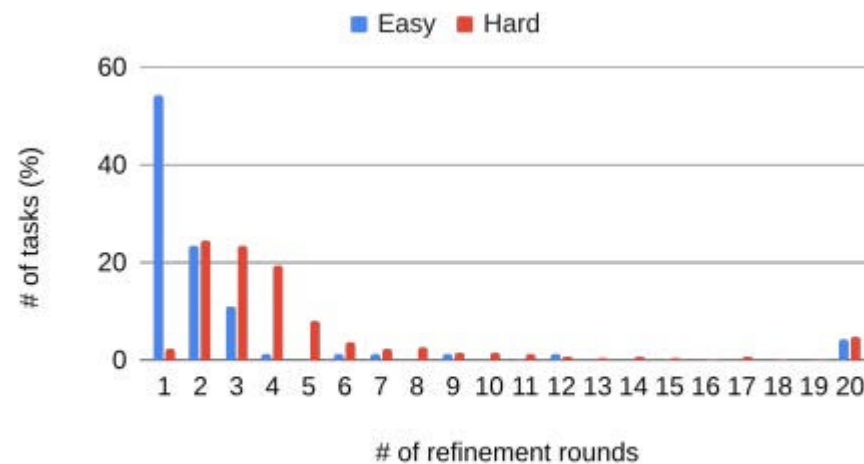
One additional observation from the [Google Research blog](#): DS-STAR with GPT-5 performs better on easy tasks, while DS-STAR with Gemini 2.5 Pro performs better on hard tasks. The same harness; different model strengths revealed. This is not an argument against using a strong model — it is evidence that the harness exposes model strengths rather than averaging them away.

More rounds for harder problems

Hard tasks require an average of 5.6 iterations to reach a sufficient plan. Easy tasks require 3.0 iterations. More than 50% of easy tasks finish in a single round.

The system does not over-iterate on simple problems. It allocates refinement rounds proportionally to the complexity of what it has been asked. An easy question gets one pass. A hard question that requires multi-step reasoning across multiple datasets gets five or six.

This matters practically. Token cost and latency scale with iteration count. A system that terminates early on simple tasks and persists on hard ones is behaving correctly.



(a) Distribution of number of required rounds

8. Google's example report

Appendix G of the paper contains three example DS-STAR+ outputs. Report 3 is the one I find more interesting as it touches on feature engineering for machine learning problems.

The question: *"Generate a comprehensive data preparation report for optimizing payment processing fee calculations. The report should analyze the relationships between merchant characteristics, transaction attributes, and fee structures across multiple datasets. Include analysis of data quality issues, feature engineering for fee calculation, and validation of fee rule applicability."*

The output covers:

1. Feature engineering decisions for fee calculation (which fields to join, which rules apply under which conditions)
2. Data quality issues flagged across the datasets (missing values, inconsistent encodings, rule applicability edge cases)
3. Validation of whether the fee rules in the manual can actually be applied to the given data as-is.
4. Each section cites the sub-question and code execution that grounded it.

3.2. Engineering Features for Fee Logic

The `manual.md` and `fees.json` files describe fee rules based on specific conditions. We engineered features to match these conditions directly:

- **is_intracountry Flag:** A new boolean column was created by comparing the `issuing_country` and `acquirer_country` for each transaction. This flag directly corresponds to the `intracountry` field in the `fees.json` rules and is the most critical feature for local acquiring analysis. Our analysis found **24,659** intracountry transactions in the dataset [14].
- **Risk Indicators:** We identified columns that serve as proxies for transaction risk, which often correlates with higher fees.
 - **ACI 'G' Transactions:** Transactions where `aci` is 'G' represent a high-risk "Card Not Present - Non-3-D Secure" segment. There are **25,463** such transactions [28].
 - **Fraudulent Disputes:** The `has_fraudulent_dispute` flag marks transactions that led to a dispute. Overall, **7.79%** of transactions fall into this category [11]. Further analysis reveals that transactions with fraudulent disputes have a slightly higher average amount (€93.36) compared to non-disputed ones (€91.82) [a]. The two most common device types for these disputed transactions are 'Other' and 'Windows', which together account for over 10,000 fraudulent disputes [c].

The final prepared dataset contains a single row per transaction with all the necessary attributes from both the transaction itself and its associated merchant.

Example snapshot of a section of the report (page 37). As a Data Scientist, this sort of report would massively help me look into specific areas of the data.

9. Limitations

3 limitations I personally think are worth naming.

1. **No official open-source release.** Google has not released an official implementation. [JulesLscx/DS-Star](#) is a community re-implementation — faithful and well-documented, but not maintained by the paper authors.
2. **Integration with Claude Code is an open problem.** The natural question for Claude Code users is: can DS-STAR patterns be applied within a Claude Code workflow? The [MCP documentation](#) shows that Claude Code can connect to external data sources via Model Context Protocol servers. A DS-STAR-style harness could, in principle, be

implemented as a set of skills or agents too. No one has done this yet in a published, production-tested form.

3. **The Analyzer is powerful but basic.** The current Analyzer generates a Python script and runs it — essentially a programmatic `describe()`. Tools like `ydata-profiling` (formerly `pandas-profiling`) produce far richer data quality summaries: distribution plots, correlation matrices, missing value heatmaps, categorical encoding warnings. Whether a richer Analyzer context would push performance further is a real research question the paper does not address.
4. **The naming is slightly misleading.** From what the agent system produces, it looks like more a product data science agent. I can't see this agent producing machine learning solutions and iterating on them based on a target metric. I can see other open source solutions, like [Karpathy's autoresearch tool](#), being focused on this sort of problem.

Closing thoughts

DS-STAR works because of its harness.

Building a system that profiles the data before planning, plans one step at a time, verifies each step against execution output, and corrects the specific step that was wrong rather than continuing past it. Seven focused agents, each doing one thing, each constrained by a short and specific prompt.

The insight that sticks with me is the ablation result on the Analyzer. The biggest performance drop comes from removing the step that generates a description of the data files. Not the verifier. Not the router. The description.

Before any planning starts, DS-STAR reads the data and builds a model of it. Everything after that is grounded in that model. Skip it, and the rest of the pipeline is planning in the dark.

For anyone thinking about how to build more reliable data science workflows on top of LLMs: the lesson is not to add more capability to one agent. The lesson is to split the work into smaller, verifiable steps and to make sure the first step is understanding the data.

Now, I want to hear from you

- Have you tried using an LLM agent for real data science work — not just exploratory analysis, but inference, validation, quality assessment? What did it get wrong?
- The ablation result puts the Analyzer as the highest-importance component. Does that match your intuition about where agentic data science systems fail?
- If you could add one module to DS-STAR's pipeline — beyond the seven it already has — what would it be?

Further reading

If you are interested in more content, here is an article capturing all my written blogs!



All my articles, one convenient place

JOSE PARREÑO GARCIA · OCTOBER 15, 2024

[Read full story](#)

References

1. Nam, J., Yoon, J., Chen, J., Sinha, R., & Pfister, T. (2025). [DS-STAR: Data Science Agent for Solving Diverse Tasks across Heterogeneous Formats and Open-Ended Queries](#). arXiv:2509.21825v4.
2. Yoon, J., & Nam, J. (2025). [DS-STAR: A state-of-the-art versatile data science agent](#). Google Research Blog.
3. Martin Iglesias et al. (2025). [DABStep: Data Agent Benchmark for Multi-step Reasoning](#). Hugging Face Blog.
4. Lai et al. (2025). [KramaBench: A Benchmark for AI Systems on Data-to-Insight Pipelines over Data Lakes](#). arXiv:2506.06541.

Thanks again for supporting the Senior Data Science Lead newsletter.

Your support makes it possible for me to keep writing openly, while also investing time in the more structured, hands-on work — courses, templates, and tools — that help people apply these ideas in practice. You can find them [here](#).



50+ Data Science Templates

JOSE PARREÑO GARCIA · APRIL 8, 2025

[Read full story](#)



Master Data Storytelling

JOSE PARREÑO GARCIA · NOVEMBER 27, 2025

[Read full story](#)

This newsletter exists in its current form because of readers like you. I am genuinely grateful for that.

Thank you.

 LIKE

 COMMENT

 RESTACK

© 2026 Jose Parreño Garcia
548 Market Street PMB 72296, San Francisco, CA 94104
[Unsubscribe](#)

Get the app

 Start writing